Available online at www.sciencedirect.com
SCIENCE DIRECT®
Information Processing Letters 92 (2004) 253–256
Information Processing Letters
www.elsevier.com/locate/ipl

Compressing arrays by ordering attribute values

Owen Kaser

Computer Science and Applied Statistics, University of New Brunswick, Saint John, Canada

Received 15 May 2003

Available online 11 September 2004

Communicated by F. Dehne

Keywords: Data structures; Databases; On-line analytical processing; Storage

1. Introduction

In On-line Analytical Processing (OLAP) and other areas, one often works with large, high-dimensional data sets whose bulk makes for slow processing. This paper considers how OLAP data is often stored in chunked arrays. Two steps, normalization and chunk selection, are usually done somewhat arbitrarily. However, better storage utilization can be obtained by doing these steps more carefully.

In Multidimensional OLAP (MOLAP), storage of a $(k+1)$-attribute relation $\mathcal{R} \subseteq \mathbb{R} \times D_1 \times D_2 \times \cdots \times D_k$ commonly involves a k-dimensional array, or *cube*. The first attribute, the *measure value*, is functionally determined by the k remaining *dimensional* attributes. For storage, one first *normalizes* $\mathcal{R}'s$ dimensional attributes by mapping attribute values to integers. E.g., if $D_1 = \{\texttt{Red}, \texttt{White}\}$, one might map `Red` to 1 and `White` to 2, or vice versa. In general, the mapping σ_i of the ith attribute is a bijection from D_i to $\{1, 2, \ldots, |D_i|\}$. (For simplicity, assume that $D_i = \pi_i(\mathcal{R})$; every value in D_i occurs in $\mathcal{R}$.)

Consider the tuple $(m, v_1, v_2, \ldots, v_k) \in \mathcal{R}$. We can treat each of the k attributes as a dimension of a multidimensional array A; the integer values $\sigma_1(v_1), \ldots, \sigma_k(v_k)$ provide an index into A, and the measure value m is stored as $A[\sigma_1(v_1), \sigma_2(v_2), \ldots, \sigma_n(v_k)]$. Some of the indices within the array do not correspond to any tuple of the relation; these store a flag indicating "no measure value". The array is *sparse* if there are far fewer measure values than flagged values; otherwise the array is *dense*.

If a relation can be stored in a dense cube, there is a space saving: no memory is occupied by dimensional values, which can be reconstructed from the index storing the measure value. However, many relations cannot be stored in a dense cube; for these, some sort of sparse-array storage is required. Most such techniques encode and store the dimensional attributes along with the measure value: effectively, they store each original tuple.

Since applications may use data of nonuniform sparseness, there may be some sub-arrays that are

E-mail address: owen@unbsj.ca (O. Kaser).

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2004.08.003

dense, even if the overall array is sparse. In fact, the use of "chunking" [1,2] encourages us to distinguish between sparse and dense chunks, which are encoded differently. This permits both space efficiency and the ability to store any relation. Cheung et al. [3] describe a system that uses these observations and develop algorithms to identify the dense chunks and process them efficiently.

To gain more space efficiency we promote dense chunks, by exploiting the system's freedom to normalize attributes and set chunk boundaries. Although this problem was mentioned by Deshpande et al. [4], this paper formalizes the problem and establishes its NP-hardness (Section 3). Finally, it considers related problems, such as forming a single dense chunk (Section 4). First, though, we must discuss some required background.

2. Background

This section reviews the chunked-array representation and explains how normalization and chunking interact to affect chunk density.

2.1. Chunked arrays

Large multidimensional arrays are often broken into *chunks* for storage and processing [1]. Consider some normalized k-dimensional array A, whose dimensionality is $|D_1| \times |D_2| \times \cdots \times |D_k|$. Chunks can be formed by breaking each D_i into several ranges. Within A, two positions are in the same chunk iff, in every dimension, they fall within the same range. (See Fig. 1.) In memory or on disk, values within a chunk are stored consecutively, in some order. Also, the dense/sparse classification can be extended to chunks in the obvious manner. Therefore, an array may be represented as a mixture of sparse and dense chunks, but since there is some per-chunk storage overhead, we do not want too many chunks.

2.2. Normalization and boundary setting affect chunk density

Normalization arbitrarily imposes a linear ordering on the values of each dimensional attribute, and the particular linear orderings will affect measure-value placement within the array. With chunking, some orderings may group most of the measure values together into a few dense chunks. Since a sufficiently dense chunk can store each value more efficiently than a sparse chunk, linear orderings that cluster measure values are better than linear orderings that disperse those values evenly throughout the array. (See Fig. 2.)

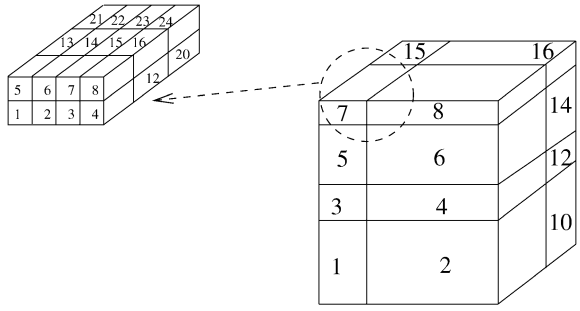

Fig. 1. 3-D cube, divided into sixteen chunks that are numbered in row-major order. Chunk 7 is itself a $4 \times 2 \times 3$ array whose 24 cells are numbered in row-major order and are stored contiguously.

Once the dimensional attributes have been normalized, the chunk boundaries must be set. For simplicity, assume that the number of chunks along each dimension has been specified, and it remains to decide exactly where the boundaries should be set. Ideally, boundaries would be set so that each cluster of measure values fits neatly into a small number of dense chunks, none of which have a large sparse region. (But see the right part of Fig. 2.)[1]

3. Formalization and complexity

Unfortunately, it is NP-hard to solve our problem optimally, even in a very simple two-dimensional case. First, though, we must formalize a special case of the problem. Since it is two-dimensional, it is easy to explain in terms of the rows and columns of a matrix, and in terms of dividing a matrix into chunks (submatrices/blocks). Parameter D represents the cost of storing a (possibly empty) index that is within the

[1] In the two-dimensional case, chunking corresponds to imposing a rectilinear grid. Also, note that this data could fit into four very dense chunks if it were better normalized, or there is also a normalization that leads to a chunk of roughly 50 percent density that contains about half the measure values.

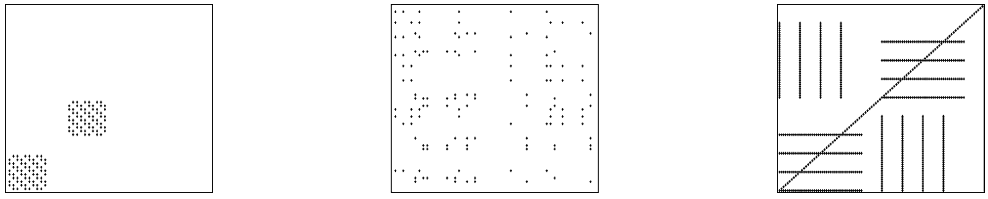

Fig. 2. Normalization affects chunk density and ability to set chunk boundaries, even in two dimensions. Left: Normalization that promotes two dense chunks. Middle: Random normalization of the same data has no obvious dense chunks. Right: Intuitively, there should be sixteen dense chunks, but the chunking rules require that each of these vertical or horizontal dense chunks be split into four pieces.

dense chunk, whereas S is the cost of storing a non-empty index within a sparse chunk.

Problem 1 (*2-D 4-chunk compression* (*2D4C*)). Given a $0/1$ matrix $\mathbf{P}$ and positive integers B, D, S, can the rows and then the columns of $\mathbf{P}$ be permuted, and then divided into 4 ($= 2 \times 2$) chunks, such that the *storage cost* is at most B? The storage cost of

$$\begin{pmatrix} \mathbf{P}_{1,1} & \mathbf{P}_{1,2} \\ \mathbf{P}_{2,1} & \mathbf{P}_{2,2} \end{pmatrix} = \sum_{1 \leqslant i,j \leqslant 2} \min\bigl\{ D \cdot \text{size}(\mathbf{P}_{i,j}), S \cdot \text{entries}(\mathbf{P}_{i,j}) \bigr\},$$

where $\text{size}(\mathbf{M}) = xy$ if matrix $\mathbf{M}$ is of order $y \times x$ and $\text{entries}(\mathbf{M})$ denotes the number of ones in $\mathbf{M}$.

This problem is shown NP-hard using a reduction from the following problem, which Peeters has shown NP-hard [5]. Recall that a *biclique* is a complete bipartite graph; the problem asks whether a given bipartite graph contains a biclique with many edges.

Problem 2 (*Maximum Edge Biclique* (*MEB*)). Given a bipartite graph $G = (V_0, V_1, E)$ and a positive integer K, are there two subsets $V_0' \subseteq V_0$ and $V_1' \subseteq V_1$ such that $|V_0'| \cdot |V_1'| \geqslant K$ and such that $v \in V_0'$ and $w \in V_1'$ implies that $(v, w) \in E$?

Theorem 1. *2D4C is* NP*-complete.*

Proof. The problem is clearly in NP. A reduction from MEB establishes NP-hardness.

Given an instance of MEB, form an instance of 2D4C by adjoining the bipartite adjacency matrix of G to an all-zero row and to an all-zero column. Specifically, number the vertices of V_0 arbitrarily as $v_1^{(0)}, v_2^{(0)}, \ldots, v_y^{(0)}$ and number the vertices of V_1 arbitrarily as $v_1^{(1)}, v_2^{(1)}, \ldots, v_x^{(1)}$. Then form the matrix $\mathbf{P}$, whose rows correspond to vertices in V_0 and whose columns correspond to vertices in V_1.

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & e_{1,1} & e_{1,2} & \cdots & e_{1,x} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & e_{y,1} & e_{y,2} & \cdots & e_{y,x} \end{pmatrix},$$

where $e_{i,j} = \begin{cases} 1, & \text{if } (v_i^{(0)}, v_j^{(1)}) \in E, \\ 0, & \text{otherwise.} \end{cases}$

Finally, set D to $2|E|$, set S to $1 + D$, and set B to $K \cdot D + (|E| - K) \cdot S$.

Suppose that the answer is "yes" for the original instance of MEB. Permute the rows of $\mathbf{P}$ so the rows corresponding to vertices in V_0' are topmost. Then permute the columns, so that columns corresponding to vertices in V_1' are leftmost. The submatrix $\mathbf{P}_{1,1}$ is entirely ones, because of the biclique in MEB. It contains $K' > K$ ones, and thus the remaining three submatrices contain $|E| - K'$ ones. Therefore, the storage cost is at most B.

Conversely, suppose the answer is "yes" for the instance of 2D4C. Meeting bound B implies storing at least K ones in completely dense chunks: since $S > D$ there cannot be more than $|E| - K$ stored the more expensive way. Thus, there is at least one dense chunk; without loss of generality, suppose that $\mathbf{P}_{1,1}$ is dense. Also, $\mathbf{P}_{1,1}$ is *completely* dense: it contains no zeros. To see this, note that storing a zero in $\mathbf{P}_{1,1}$ requires paying $2|E|$ ($= D$). The most benefit that could be obtained is the ability to store *all* $|E|$ ones at the slightly cheaper rate of D. But since $S = D + 1$, the total benefit does not outweigh the cost of storing a zero in a dense chunk; the bound B enforces that all dense chunks must be completely dense. A completely dense chunk corresponds to a biclique; thus, the instance of

MEB has at least K edges belonging to bicliques. It remains to show that there is only one biclique, but this is enforced by the column and row of zeros adjoined to **P**. No matter how columns and rows are permuted, at most one chunk ($\mathbf{P}_{1,1}$) can be completely dense. Therefore, from $\mathbf{P}_{1,1}$ we can find a biclique with at least K edges in the instance of MEB. □

4. Discussion

Several issues remain. First, will reasonable practical restrictions remove the problem's NP-hardness? Second, what are some heuristics, and can they obtain useful compression? Third, where else does the problem arise?

Storage managers may impose extra restrictions; perhaps all chunks must be of a uniform shape. See Kaser and Lemire [7] for an analysis and experimental results for this case: though Theorem 1 does not apply, this problem is also NP-hard. Yet simple heuristics yielded useful compression.

Consider instead the restriction that exactly one chunk is stored densely and note that the previous proof implies that the problem remains NP-hard. Now the values of each dimension merely need be classified as sparse or dense, and point $(x_1, \ldots, x_k)$ in k-dimensional space belongs to the dense chunk iff each coordinate x_i has been classified as dense. The specific ordering σ_i of the dimension's values is insignificant, providing σ_i makes all dense values consecutive. Yet by ordering dense values first, we get an especially simple storage implementation. At most k comparisons are required, since one endpoint of each dense range is 1. Whether "real world" data sets gain much space compression, despite this drastic restriction, is addressed by Kaser [6]. For data sets of 1–10% density, positive results are reported for such simple heuristics as "σ_i orders D_i by decreasing frequency of occurrence in $\mathcal{R}$".

Finally, there is clearly some applicability (beyond MOLAP) whenever large multidimensional arrays are stored and processed. It may be useful to consider problem variants specific to these other areas, or explore whether the broad idea, "reorder to cluster for storage", can be applied when $X > 2$ degrees of sparseness lead to X preferred storage mechanisms.

Acknowledgement

Thanks to Daniel Lemire for helpful discussions.

References

[1] S. Sarawagi, M. Stonebraker, Efficient organization of large multidimensional arrays, in: Proc. Eleventh Internat. Conf. on Data Engineering, 1994.

[2] Y. Zhao, P.M. Deshpande, J.F. Naughton, An array-based algorithm for simultaneous multidimensional aggregates, in: SIGMOD'97, pp. 159–170.

[3] D.W.-L. Cheung, B. Zhou, B. Kao, H. Kan, S.D. Lee, Towards the building of a dense-region-based OLAP system, Data and Knowledge Engineering 36 (1) (2001) 1–27.

[4] P. Deshpande, K. Ramasamy, A. Shukla, J. Naughton, Caching multidimensional queries using chunks, in: SIGMOD'98, pp. 259–280.

[5] R. Peeters, The maximum-edge biclique problem is NP-complete, Research Memorandum 789, Faculty of Economics and Business Administration, Tilberg University (2000). URL: http://econpapers.hhs.se/paper/dgrkubrem/2000789.htm (checked 20/01/2004).

[6] O. Kaser, Compressing MOLAP arrays by attribute-value reordering: an experimental analysis, Tech. Rep. TR-02-001, Dept. of CS and Appl. Stats, Univ. of New Brunswick, Saint John, Canada (August 2002).

[7] O. Kaser, D. Lemire, Attribute-value reordering for efficient hybrid OLAP, in: ACM DOLAP'03, 2003, pp. 1–8.